14 — FastAPI Service Setup Plan

Task 14 — Sprint 3 — Foundation & Planning

Planning only — no implementation. Production/cloud constraints: `16_Production_and_Cloud_Readiness.md` (stateless service, ready probes, env secrets, one image). This document is the API contract that Task 30 (Sprint 5) freezes and the Sprint 9 frontend builds against.

1. Service Overview

Property	Value
Framework	FastAPI (ADR-004 stack)
Server	uvicorn, `app.main:app`
Base path	`/api/v1`
Content type	`application/json` (plus `text/event-stream` for streaming)
Auth	Bearer JWT — **stub in Sprint 4**; **signup + login** in Sprint 8 (Tasks 43/44)
Docs	`/docs` (Swagger), `/redoc`, `/openapi.json`
Statelessness	fully stateless; session state in Postgres (ADR-006)
Versioning	additive changes only after the Sprint 5 freeze; breaking changes require `/api/v2`

2. Endpoint Inventory

Method	Path	Purpose	Auth	Sprint
`POST`	`/api/v1/query`	Submit a financial query, get a grounded answer / refusal / escalation	required	4
`POST`	`/api/v1/query/stream`	Same, streamed as SSE (progress + token deltas)	required	9
`GET`	`/api/v1/sessions/{session_id}/turns`	Recent turns for follow-up context	required	5
`DELETE`	`/api/v1/sessions/{session_id}`	End a session, purge its turns	required	5
`GET`	`/api/v1/documents`	List the 7 policy documents with metadata	required	5
`GET`	`/api/v1/escalations`	List escalations for the caller's scope	risk_manager, compliance	7

Method	Path	Purpose	Auth	Sprint
GET	/api/v1/escalations/{escalation_id}	Full escalation envelope	risk_manager, compliance	7
POST	/api/v1/auth/signup	Register a new user (email + password)	none	8
POST	/api/v1/auth/login	Login → JWT (alias: /auth/token kept for compatibility)	none	8
GET	/api/v1/auth/me	Current principal (id, email, role, portfolio scope)	required	8
PATCH	/api/v1/admin/users/{user_id}	Admin sets role / portfolio scope	admin	8
GET	/health	Liveness	none	4
GET	/health/ready	Readiness: DB, vector index, LLM reachability	none	4
GET	/metrics	Prometheus metrics	internal	4
POST	/admin/reindex	Re-ingest the policy corpus	admin	4
GET	/admin/traces/{trace_id}	Span tree + evidence bundle for a past request	admin, compliance	4
GET	/admin/eval/latest	Latest SLO report	admin	5
GET	/admin/templates	SQL template registry (ids, descriptions, slots)	admin	4
GET	/api/v1/screens/fields	Screen field catalogue (whitelist)	required	4
POST	/api/v1/screens/run	Run stock screen definition	required	4
POST	/api/v1/screens/from-nl	NL → screen + optional run	required	6
GET/POST	/api/v1/screens	List / save screens	required	8
GET/PUT/DELETE	/api/v1/screens/{id}	Saved screen CRUD	owner	8
GET	/api/v1/screens/{id}/export.csv	Export results	owner	9

3. Core Contract — `POST /api/v1/query`

3.1 Request

```
{
  "query": "Are any assets exceeding the allowed risk thresholds for our portfolio strategy?",
  "session_id": "sess_9f3a...",           // optional; created if absent
  "options": {                            // all optional
    "max_results": 5,                     // retrieval top-k override, 1..10
    "include_evidence": false,            // return raw evidence bundle (admin/debug)
    "explain_routing": true               // include routing rationale
```

```
  }
}
```

Headers: `Authorization: Bearer <jwt>` (required) — `Idempotency-Key: <uuid>` (optional) — `X-Request-Id` (optional; echoed).

Validation: `query` 3–2000 chars, non-empty after strip; `session_id` matches `^sess_[a-zA-Z0-9]{8,}$`; unknown fields rejected (`extra="forbid"`).

3.2 Response — 200 (answered)

```
{
  "trace_id": "trc_7c21ab...",
  "session_id": "sess_9f3a...",
  "outcome": "answered",                   // answered | refused | escalated |
insufficient_evidence
  "answer": "**Policy requirement**\nExposure to emerging market assets is capped at 15% ...",
  "citations": [
    { "type": "doc",
      "doc_id": "02_Risk_Management_Exposure_Policy",
      "doc_title": "Risk Management & Exposure Policy",
      "section": "Exposure Limits",
      "page": 14,
      "chunk_id": "chunk_02_014_3",
      "score": 0.87 },
    { "type": "sql",
      "query_id": "risk_logs.by_score_threshold",
      "tables": ["asset_risk_logs"],
      "params": { "min_score": 70 },
      "row_count": 2,
      "executed_at": "2026-09-02T11:04:18Z" }
  ],
  "routing": {
    "category": "C3_HYBRID",
    "category_label": "Hybrid Risk Evaluation",
    "path": ["retrieval", "portfolio_eval", "risk", "synthesis"],
    "agents_used": ["router", "retrieval", "portfolio_eval", "risk", "synthesis"],
    "confidence": 0.92,
    "rationale": "Requires policy thresholds compared against observed risk scores."
  },
  "risk": {
    "level": "High",
    "rationale": "Two holdings carry risk scores above the 80 escalation threshold ...",
    "any_escalation_flag": true,
    "breached_thresholds": [
      { "name": "Emerging market exposure cap", "policy_value": 15.0, "observed_value": 18.4,
        "unit": "percent", "source_chunk_id": "chunk_02_014_3",
        "source_query_id": "holdings.by_sector" }
    ],
    "flagged_assets": [
      { "asset_name": "Emerging Market ETF", "risk_id": 92, "risk_score": 91, "escalation_flag":
true }
    ]
  },
  "escalation": {
    "required": false,
```

```
    "triggers": [],
    "reason": null,
    "recommended_reviewer": null,
    "escalation_id": null
  },
  "confidence": 0.81,
  "evidence_summary": { "doc_chunks": 4, "sql_queries": 2, "sql_rows": 8, "market_indicators": 0
},
  "gaps": [],
  "disclaimer": "Based on synthetic/educational data for demonstration purposes. This is not
investment advice. All outputs require review by a qualified human analyst.",
  "latency_ms": 3120,
  "token_usage": { "prompt": 5820, "completion": 640, "total": 6460 },
  "cost_usd": 0.0214,
  "model_info": { "router": "gpt-4o-mini", "agent": "gpt-4o" }
}
```

3.3 Response — 200 (escalated)

Same envelope, with:

```
{
  "outcome": "escalated",
  "answer": "**Escalated for human review**\n**Reason:** E1 — the request asks the system to
override a risk assessment ...",
  "escalation": {
    "required": true,
    "triggers": ["E1", "E2"],
    "trigger_details": [
      { "id": "E1", "name": "Override / bypass intent",
        "description": "The request instructs the system to bypass a risk control." },
      { "id": "E2", "name": "Flagged asset in scope",
        "description": "An asset in scope has escalation_flag = TRUE." }
    ],
    "reason": "Request seeks to bypass an active escalation flag on Emerging Market ETF.",
    "recommended_reviewer": "Risk Committee",
    "escalation_id": "esc_4b81f2"
  },
  "risk": { "level": "Critical", ... },
  "confidence": 1.0
}
```

> An escalated response contains **no answer to the original question** and **no description of how the requested bypass could be performed**. `outcome` is the field a client must branch on — never string-match the answer text.

3.4 Response — 200 (refused / insufficient evidence)

`outcome` is `refused` (guardrail: non-advice, or an override refusal without escalation context) or `insufficient_evidence`. `answer` holds the rendered template; `citations` may be empty; `gaps` describes what was missing; `confidence` is low.

3.5 Errors

Status	Condition	Body
400	validation failure	`{error: {code:"invalid_request", message, field}}`
401	missing/invalid token	`{error: {code:"unauthenticated", ...}}`
403	RBAC scope violation detected	`{error: {code:"forbidden", message:"Requested data is outside your authorized scope", trace_id}}`
404	unknown session / trace / escalation	`{error: {code:"not_found", ...}}`
422	semantically unprocessable query	`{error: {code:"unprocessable", ...}}`
429	rate limit	`Retry-After` header
503	LLM or DB unavailable, or synthesis could not produce a cited answer	`{error: {code:"service_degraded", message, trace_id}}`
504	request exceeded 2— the category budget	`{error: {code:"timeout", trace_id, partial_routing}}`

All error bodies carry `trace_id`. **No error body ever contains an uncited partial answer.**

---

4. Pydantic Schema Plan (`app/schemas/`)

```
# request.py
class QueryOptions(BaseModel):
    max_results: int = Field(5, ge=1, le=10)
    include_evidence: bool = False
    explain_routing: bool = True

class QueryRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")
    query: str = Field(..., min_length=3, max_length=2000)
    session_id: str | None = Field(None, pattern=r"^sess_[a-zA-Z0-9]{8,}$")
    options: QueryOptions = QueryOptions()

# citations.py
class DocCitation(BaseModel):
    type: Literal["doc"] = "doc"
    doc_id: str; doc_title: str; section: str; page: int; chunk_id: str; score: float

class SqlCitation(BaseModel):
    type: Literal["sql"] = "sql"
    query_id: str; tables: list[str]; params: dict; row_count: int; executed_at: datetime

Citation = Annotated[DocCitation | SqlCitation, Field(discriminator="type")]

# response.py
class RoutingInfo(BaseModel):
    category: QueryCategory; category_label: str; path: list[str]
    agents_used: list[str]; confidence: float; rationale: str | None

class BreachedThreshold(BaseModel):
    name: str; policy_value: float; observed_value: float; unit: str
    source_chunk_id: str | None; source_query_id: str | None
```

```
class FlaggedAsset(BaseModel):
    asset_name: str; risk_id: int; risk_score: int; escalation_flag: bool

class RiskBlock(BaseModel):
    level: RiskLevel | None; rationale: str | None; any_escalation_flag: bool
    breached_thresholds: list[BreachedThreshold] = []
    flagged_assets: list[FlaggedAsset] = []

class TriggerDetail(BaseModel):
    id: str; name: str; description: str

class EscalationBlock(BaseModel):
    required: bool; triggers: list[str] = []; trigger_details: list[TriggerDetail] = []
    reason: str | None = None; recommended_reviewer: str | None = None
    escalation_id: str | None = None

class QueryResponse(BaseModel):
    trace_id: str; session_id: str
    outcome: Literal["answered", "refused", "escalated", "insufficient_evidence"]
    answer: str
    citations: list[Citation]
    routing: RoutingInfo
    risk: RiskBlock
    escalation: EscalationBlock
    confidence: float
    evidence_summary: dict[str, int]
    gaps: list[str] = []
    disclaimer: str            # constant, always populated
    latency_ms: int
    token_usage: dict[str, int]
    cost_usd: float
    model_info: dict[str, str]
    evidence: EvidenceBundle | None = None   # only when options.include_evidence and role allows
```

Schema invariants enforced by validators: 1. `disclaimer` is never empty. 2. `outcome == "answered"` requires `len(citations) >= 1`. 3. `outcome == "escalated"` requires `escalation.required is True` and a non-null `recommended_reviewer`. 4. `routing.category == "C3_HYBRID"` requires >=1 `doc` and >=1 `sql` citation. 5. `evidence` is populated only for `admin`/`compliance` roles.

---

5. Auth Stub → Real (Tasks 43/44)

Sprint 4 stub (`AUTH_MODE=stub`)

`app/api/deps.py` reads an `X-Debug-Role` header (or defaults to `analyst`) and resolves a hardcoded scope map — **available only when `APP_ENV != prod`**, asserted at startup.

```
STUB_USERS = {
  "analyst":      {"user_id": "u_analyst",  "allowed_portfolio_ids": [1, 2]},
  "risk_manager": {"user_id": "u_risk",     "allowed_portfolio_ids": "ALL"},
  "compliance":   {"user_id": "u_comp",     "allowed_portfolio_ids": "ALL"},
  "admin":        {"user_id": "u_admin",    "allowed_portfolio_ids": "ALL"},
}
```

Sprint 8 real (`AUTH_MODE=jwt`) — signup + login

New users **sign up**, then **log in** to get a JWT. Seeded admin still exists for role/portfolio assignment.

Auth database: `finance_auth_db` (ADR-020 — **not** the vector DB)

Connection: `AUTH_DATABASE_URL`. Tables: `users`, `sessions`, `turns`. `finance_risk_db` (`DATABASE_URL`) holds only finance tables + `policy_chunks`.

`users` table (`finance_auth_db`)

```
CREATE TABLE users (
  user_id         TEXT PRIMARY KEY,          -- u_<uuid>
  email           CITEXT UNIQUE NOT NULL,
  password_hash   TEXT NOT NULL,             -- argon2id / bcrypt
  full_name       TEXT,
  role            TEXT NOT NULL DEFAULT 'analyst'
                  CHECK (role IN ('analyst','risk_manager','compliance','admin')),
  portfolio_ids   INT[] NOT NULL DEFAULT '{}',  -- empty = no portfolio data until assigned
  status          TEXT NOT NULL DEFAULT 'active'
                  CHECK (status IN ('active','disabled')),
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  last_login_at   TIMESTAMPTZ
);
```

`POST /api/v1/auth/signup` (public)

Request:

```
{ "email": "new.analyst@firm.com", "password": "********", "full_name": "Ada Analyst" }
```

Rules: - email unique (409 if taken); password min 8 chars - creates `role=analyst`, `portfolio_ids={}` (no data until admin assigns), `status=active` - returns `{ user_id, email, role }` — **does not** auto-login (client calls login next) - rate-limited; never echo password; hash only stored

`POST /api/v1/auth/login` (public) — same body as legacy `/auth/token`

Request:

```
{ "email": "new.analyst@firm.com", "password": "********" }
```

Response:

```
{
  "access_token": "<jwt>",
  "token_type": "bearer",
  "expires_in": 3600,
  "user": { "user_id": "u_...", "email": "...", "role": "analyst", "portfolio_ids": [] }
}
```

JWT claims: `{ sub, email, role, portfolio_ids, exp }` (HS256). `get_current_user` verifies signature/expiry → `Principal`. **Token scope is authoritative** — request body cannot widen it.

Admin assignment

`PATCH /api/v1/admin/users/{user_id}` with `{ "role"?, "portfolio_ids"?, "status"? }` so a new signup can receive portfolios (e.g. `[1,2]`) or a higher role. Without assignment, analyst queries that need portfolio SQL return empty/scoped results — never other users' data.

Frontend — React (Sprint 5–9, ADR-023)

Full spec: `19_React_Frontend_Plan.md`. - **First pages:** `/login` and `/signup` only for unauthenticated users. - **After login:** `/app/*` (query, screens, escalations) — all protected. - JWT in `localStorage` for capstone dev; `Authorization: Bearer` on every protected API call. - 401 → clear token, redirect `/login`.

RBAC matrix (`app/core/security.py`)

Role	/query	Portfolio scope	/escalations	/admin/*	Evidence bundle
analyst	✅	assigned portfolios only	own escalations	✅	→
risk_manager	✅	all portfolios, all risk logs	all	✅	→
compliance	✅	all + audit log	all	traces only	✅
admin	✅	all	all	✅	→

Enforcement point is the SQL executor (ADR-014), not the prompt and not the response layer.

---

6. Health, Metrics, Admin

`GET /health` → 200

```
{"status":"ok","version":"0.4.0","uptime_s":3821}
```

`GET /health/ready` → 200 / 503

```
{"status":"ready",
 "checks":{"database":{"ok":true,"latency_ms":4},
           "vector_index":{"ok":true,"chunk_count":712},
           "llm":{"ok":true,"latency_ms":180,"model":"gpt-4o"},
           "templates":{"ok":true,"count":18}}}
```

503 if any check fails. Used as the container readiness probe.

`GET /metrics` (Prometheus)

Metric	Type	Labels
`frii_requests_total`	counter	`category`, `outcome`, `role`
`frii_request_latency_seconds`	histogram	`category`
`frii_node_latency_seconds`	histogram	`node`
`frii_escalations_total`	counter	`trigger`
`frii_guardrail_blocks_total`	counter	`attack_type`
`frii_rbac_violations_total`	counter	— (must stay 0)
`frii_citations_per_answer`	histogram	`category`
`frii_uncited_answers_total`	counter	— (must stay 0)
`frii_tokens_total`	counter	`agent`, `kind`
`frii_cost_usd_total`	counter	`agent`
`frii_sql_queries_total`	counter	`query_id`, `status`
`frii_retrieval_score`	histogram	—

`GET /admin/traces/{trace_id}`

Returns: span tree with per-node durations — routing decision and confidence — evidence bundle (`chunk_id`s and `query_id`s; chunk text only for `admin`) — escalation triggers evaluated with their outcomes — token usage and cost per node — errors recovered. This endpoint is what makes UC-17 (audit review) possible.

---

7. Application Structure (recap from `05`)

```
app/
├── main.py            # create_app(): middleware, routers, OTel instrumentation, startup checks
├── api/
│   ├── deps.py        # get_current_user, get_scope, get_graph, get_db
│   ├── routes_query.py
│   ├── routes_sessions.py
│   ├── routes_health.py
│   └── routes_admin.py
├── core/              # config, logging, tracing, llm, security
├── graph/             # state, build, nodes/
├── prompts/           # markdown prompts (13)
├── rag/               # ingest, chunking, store, retriever
├── sqlq/              # schema.sql, templates, executor
├── guardrails/        # input_filters, escalation_rules, output_validators
├── memory/            # session
└── schemas/           # request, response, evidence, citations
```

Middleware order (outermost → innermost)

1. `TraceIdMiddleware` — generate/propagate `trace_id`, bind it to the log context
2. OpenTelemetry FastAPI instrumentation

3. `RequestLoggingMiddleware` — one structured line per request on completion
4. CORS (frontend origin only, from Sprint 5)
5. `RateLimitMiddleware` — per-principal token bucket (Sprint 8)
6. `ErrorHandlerMiddleware` — map exceptions to the error envelope, always attach `trace_id`

Startup checks (fail fast)

config validation — DB reachable and `vector` extension present — `policy_chunks` non-empty — template registry loads and every template compiles — every prompt file present — read-only role confirmed (an INSERT attempt must fail) — `AUTH_MODE=stub` rejected when `APP_ENV=prod`.

---

8. Streaming Contract (`/query/stream`, Sprint 9)

SSE event sequence:

```
event: routing    data: {"category":"C3_HYBRID","path":[...],"confidence":0.92}
event: progress   data: {"node":"retrieval","status":"complete","elapsed_ms":680}
event: progress   data: {"node":"portfolio_eval","status":"complete","elapsed_ms":540}
event: citation   data: {"type":"doc","doc_title":"...","page":14,...}
event: token      data: {"text":"**Policy requirement**\n"}
...
event: escalation data: {"required":true,"triggers":["E2"],"recommended_reviewer":"Risk
Committee"}
event: done       data: {<the full QueryResponse envelope>}
```

Rule: if escalation fires mid-stream, emit the `escalation` event and **stop token streaming immediately**; the client must discard partial answer text. The `done` envelope is always authoritative.

---

9. Non-Functional Requirements

Concern	Requirement
Latency	per-category budgets from ADR-010, measured server-side; `latency_ms` returned in every response
Timeouts	node `NODE_TIMEOUT_MS`; request timeout = 2— category budget
Concurrency	async throughout; DB pool 2–10; no blocking calls in the event loop
Rate limiting	30 req/min per principal (Sprint 8)
Payload caps	request 4KB; response evidence bundle capped at 200 rows / 10 chunks
Idempotency	`Idempotency-Key` replays the stored envelope within the session TTL; never re-runs the graph, never creates a second escalation record
CORS	explicit frontend origin only; no wildcard
Security headers	`X-Content-Type-Options`, `X-Frame-Options`, `Strict-Transport-Security` (prod)
Secrets	env only; never logged, never returned, never in traces
Logging redaction	query text logged; PII-scan hook reserved; chunk text never logged

Concern	Requirement
Graceful shutdown	drain in-flight requests, close pools, flush spans

10. Frontend-Facing Contract Notes (for Sprint 9)

UI need	Field to use
Show the answer	`answer` (markdown; render inline citation markers as chips)
Clickable citations	`citations[]` — `doc_title` + `section` + `page`, or `query_id` + `row_count`
"How was this answered?" panel	`routing` (category label, agents used, confidence, rationale)
Risk badge	`risk.level`, `risk.any_escalation_flag`
Breach warnings	`risk.breached_thresholds[]` (policy vs. observed)
Escalation banner	`escalation.required`, `triggers`, `recommended_reviewer`, `escalation_id`
Restricted-action UX (Task 51)	branch on `outcome`; for `escalated`/`refused` **hide any action affordance** and show the reviewer routing
Confidence indicator	`confidence`
Partial-coverage notice	`gaps[]`
Mandatory disclaimer	`disclaimer` — must be rendered on every response, never suppressed
Debug / audit link	`trace_id` → `/admin/traces/{trace_id}` (compliance/admin only)

Contract rules for the client: branch on `outcome`, never on answer text — always render `disclaimer` — never render a partial streamed answer after an `escalation` event — treat `citations` as required for `answered`.

11. Definition of Done for Task 14 (planning)

This document is complete when: every endpoint has a method, path, auth requirement, and sprint; the `/query` request and all four response shapes are fully specified; the Pydantic plan covers every field with its invariants; the auth stub → JWT path and RBAC matrix are defined; health/metrics/admin surfaces are enumerated; and the frontend field mapping exists so Sprint 9 needs no backend redesign. **No application code is written as part of Task 14.**

> **Disclaimer.** Synthetic/educational data. Not investment advice.

11. Stock screening

See `17_Stock_Screening_Feature.md` (ADR-021). Equity tables in `finance_risk_db`; saved screens in `finance_auth_db`.

12. Production APIs

Not in capstone contract. See separate plan: `production/04_Architecture_and_APIs.md`.

13. React frontend (auth-gated)

See `19_React_Frontend_Plan.md`. Summary: - Stack: React + Vite + TypeScript + React Router - Public: `/login`, `/signup` — default landing when logged out - Protected: `/app`, `/app/screens`, `/app/escalations`, `/app/admin/*` - No feature UI accessible without valid JWT (or Sprint 4–7 auth stub during dev)